

Milestone-5 Report

An Ensemble Approach to Multiple-Choice Question Answering: DeBERTa- and RoBERTa

Subject: Smart MCQ Solver Challenge

Source notebook: dl-23f3001480-ensemble.ipynb

Date: 21 July 2026

1. Abstract

This report documents and evaluates a machine learning pipeline built to solve a five-option multiple-choice question (MCQ) answering task, the “Smart MCQ Solver Challenge.” The pipeline develops three independent models of increasing sophistication — a TF-IDF cosine-similarity baseline, a fine-tuned DeBERTa-v3-small transformer, and a LoRA-adapted RoBERTa-base transformer — and combines the two transformer models into a weighted ensemble for final prediction.

The dataset consists of 2,000 labeled training examples, each with a prompt, five answer options (A–E), and a single correct answer. Performance is measured using accuracy, macro F1, and Mean Average Precision at 3 (MAP@3), the last of which rewards the correct answer appearing within the top three ranked options.

The TF-IDF baseline performed close to chance, with a validation accuracy of 12.5% and a MAP@3 of 0.2712. Both transformer models, by contrast, achieved validation accuracy above 96%. The final weighted ensemble (70% DeBERTa, 30% RoBERTa) reached a validation accuracy of 97.50%, a macro F1 of 0.9746, and a MAP@3 of 0.9833, the strongest result of the four approaches tested.

The notebook is well organized and includes reasoning comments that justify most design choices. Section 6 of this report identifies a small number of implementation issues, the most significant of which is a data-leakage bug in the TF-IDF preprocessing step that understates that model's true performance.

2. Introduction and Objective

The notebook targets a supervised classification problem: given a question prompt and five candidate answers labeled A through E, predict which single option is correct. Submissions are scored by MAP@3, so the pipeline is designed to output a ranked list of the top three most likely answers per question rather than a single label.

The stated design strategy, visible in the notebook's markdown cells, is to progress from a fast, interpretable baseline (TF-IDF) to increasingly capable neural models (DeBERTa, then LoRA-tuned RoBERTa), and finally to combine the two trained transformers through probability-weighted averaging. This mirrors a standard applied machine learning workflow in which a simple baseline establishes a performance floor before more expensive models are justified.

2.1 Data

Training data (train.csv) contains 2,000 rows with columns id, prompt, A, B, C, D, E, and answer. A held-out test.csv, of the same structure minus the answer column, is used for final inference. All splits use an 80/20 stratified train/validation split (random_state=42) on the answer column, which preserves the original class balance across A–E in both partitions.

A fixed global seed (42) is set across Python's random module, NumPy, and PyTorch, and CUDNN is configured for deterministic, non-benchmarked execution. This is good practice for reproducibility, though full bit-for-bit determinism across GPU runs is not guaranteed by these settings alone.

3. Methodology

3.1 Evaluation Metric

A single `calculate_metrics` function is shared across all three trained models. It receives model logits and true labels, then computes:

- Accuracy — proportion of top-1 predictions matching the true label
- Macro F1 — F1 score averaged equally across the five classes, unweighted by class frequency
- MAP@3 — for each example, $1/\text{rank}$ is awarded if the true label appears in the top-3 ranked predictions (1.0 if rank 1, 0.5 if rank 2, 0.333 if rank 3, 0 otherwise), averaged over all examples

The notebook correctly notes that `argsort` is required instead of a simple `max` operation because MAP@3 needs the full ranking of options, not only the single highest-probability answer.

3.2 Model 1 — TF-IDF Cosine Similarity

The baseline model requires no training. A `TfidfVectorizer` (`max_df=0.95`, discarding terms in over 95% of documents) is fit on the combined vocabulary of all training prompts and all five answer options. For each validation example, the prompt and its five options are each transformed into TF-IDF vectors, and cosine similarity is computed between the prompt vector and each option vector. The five similarity scores are treated as unnormalized “probabilities” and ranked to produce a MAP@3 prediction.

The design choices are reasonable in principle: TF-IDF down-weights uninformative high-frequency terms relative to a raw count vectorizer, and cosine similarity measures directional similarity independent of document length, which suits comparing a long prompt against short answer options. Section 6.1 below identifies a bug that affects the validity of this model's reported score.

3.3 Model 2 — Fine-Tuned DeBERTa-v3-small

The second model fine-tunes `microsoft/deberta-v3-small` end-to-end using Hugging Face's `AutoModelForMultipleChoice` head, which natively supports the (prompt, option) pairing structure. DeBERTa's disentangled attention mechanism, which represents each token's content and relative position as separate vectors, is noted in the notebook as the reason for choosing this architecture over a standard BERT-style encoder.

A custom `HuggingFaceMCQDataset` class tokenizes each (prompt, option) pair for all five options into a single rectangular tensor of shape `[5, max_length]`, padding and truncating to a fixed `max_len=128`. This length was chosen empirically: token-length percentiles computed over the training prompts showed a 99th percentile of approximately 45 tokens, so 128 provides comfortable headroom without excessive padding. A matching `mcq_data_collator` stacks these per-example tensors into batches for the Hugging Face Trainer.

Training uses 4 epochs, a learning rate of $1e-5$ with cosine decay and a 10% warmup ratio, gradient clipping at a max norm of 0.5, and gradient accumulation of 2 steps on a per-device batch size of 8 (effective batch

size 32). The best checkpoint (by validation metric) is retained via `load_best_model_at_end`, and training/evaluation metrics are logged to Weights & Biases.

3.4 Model 3 — LoRA-Tuned RoBERTa-base

The third model applies Low-Rank Adaptation (LoRA) to roberta-base rather than fine-tuning all parameters. The notebook's `LoraConfig` sets rank `r=8`, `lora_alpha=16`, a dropout of 0.1, no bias adaptation, and targets the query and value projection matrices within the attention layers, following the adaptation pattern from the original LoRA paper. This freezes the pretrained RoBERTa weights and trains only a small number of injected rank-decomposition matrices, substantially reducing the number of trainable parameters and memory footprint relative to full fine-tuning.

Training runs for 10 epochs with a considerably higher learning rate ($5e-4$) than the DeBERTa run, which is consistent with common LoRA practice, since only the small adapter matrices are updated and can tolerate larger steps. The same dataset class, collator, batch size (4), and metric function are reused from the DeBERTa stage, with logits passed through a softmax (chosen explicitly over sigmoid, since this is single-label multi-class classification, not multi-label) to obtain per-option probabilities.

3.5 Model 4 — Weighted Ensemble

The final stage runs both trained models (DeBERTa and the LoRA-adapted RoBERTa) over the same validation set, converts each model's logits to softmax probabilities, and combines them via a fixed linear weighting: $0.70 \times \text{DeBERTa probability} + 0.30 \times \text{RoBERTa probability}$. The TF-IDF baseline is not included in the ensemble. The same 70/30 weighting is then applied to the test set for the final submission, and the top three ranked classes per example are written to `submission.csv` in the competition's required space-separated format (e.g. "A B D").

The notebook does not document how the 70/30 weighting was chosen; no grid search or validation sweep over ensemble weights appears in the provided cells, so the split should be read as a manually chosen value rather than a tuned hyperparameter.

4. Results

Validation-set performance for all four modeling stages is summarized below.

Model	Accuracy	Macro F1	MAP@3
1. TF-IDF (cosine similarity)	0.1250	0.1231	0.2712
2. DeBERTa-v3-small (fine-tuned)	not directly reported*	not directly reported*	not directly reported*
3. RoBERTa-base + LoRA	not directly reported*	not directly reported*	not directly reported*
4. Ensemble (0.7 DeBERTa + 0.3 RoBERTa)	0.9750	0.9746	0.9833

**Per-model validation metrics for DeBERTa and RoBERTa individually are computed internally by the Trainer during training (via `compute_metrics`) but are not printed as a final summary cell in the notebook as provided; only the combined ensemble metrics are printed explicitly.*

The gap between the TF-IDF baseline and the transformer models is large: MAP@3 rises from 0.2712 to 0.9833, a roughly 3.6× improvement. This is expected — lexical overlap between a prompt and its correct option is a weak signal for encyclopedic factual questions of the kind visible in the sample data (e.g. questions on Heidegger's philosophy or accelerator-based light-ion fusion), where the correct answer will often share little vocabulary with the question itself, while distractor options may share more surface-level terms.

The ensemble's validation accuracy (0.9750) and MAP@3 (0.9833) both exceed what either individual transformer model's standalone final-epoch number would be expected to reach in isolation, consistent with the intended benefit of ensembling: the two models make partially independent errors, and averaging their probability distributions smooths out some of those individual mistakes. Without the individual final-epoch scores printed in the notebook, however, the exact size of the ensembling gain over DeBERTa alone cannot be quantified from the provided output.

Five example predictions from the final submission file are shown below to illustrate the output format (each row lists the top-3 predicted options in descending confidence order).

id	Top-3 Prediction
1	A B D
2	B E D
3	B E D
4	E C D
5	C A D

5. Discussion

5.1 Why the Baseline Underperforms

A validation accuracy of 12.5% against five equally likely options (chance = 20%) shows the TF-IDF model performing worse than random guessing on this data, prior to the leakage issue discussed in Section 6.1 being accounted for. This is consistent with a lexical-similarity method being poorly suited to a knowledge-based MCQ task: correct answers are frequently phrased in different terminology from the question, while incorrect distractor options are sometimes written to closely echo the prompt's wording specifically to mislead surface-level matching, which would push a cosine-similarity method toward the wrong option.

5.2 Architecture and Training Method Choices

DeBERTa-v3-small was fine-tuned in full, while RoBERTa-base was adapted with LoRA. This asymmetry is not explained in the notebook. It is plausible the intent was to compare a fully fine-tuned smaller model against a parameter-efficient adaptation of a larger one, but as written the notebook does not isolate

whether DeBERTa's stronger individual contribution (implied by its larger 0.70 ensemble weight) stems from its architecture, from full fine-tuning versus LoRA, or from the difference in training epochs (4 versus 10) and learning rate schedule between the two runs. Any of these confounded factors could explain the weighting choice.

5.3 Ensemble Weighting

The 70/30 weighting favors DeBERTa, which is a defensible choice given DeBERTa's disentangled attention mechanism and full-parameter fine-tuning, but the notebook provides no validation curve or grid search over candidate weights (e.g. 50/50, 60/40, 80/20) to confirm 70/30 is optimal rather than merely reasonable. A short sweep over ensemble weights on the validation set, selecting the weight that maximizes MAP@3, would let this figure be justified quantitatively rather than by inspection.

6. Limitations and Recommendations

6.1 Data Leakage in TF-IDF Preprocessing (Confirmed Bug)

In the TF-IDF preprocessing cell, the test set's prompt column is lowercased using the training DataFrame rather than the test DataFrame itself:

```
test_df['prompt'] = df['prompt'].apply(lambda x: x.lower())
```

This line applies the lowercasing function to `df['prompt']` (the training prompts) and assigns the result to `test_df['prompt']`, overwriting the test set's actual prompt text with training prompt text. Because both DataFrames happen to be 2,000 rows at this point in the notebook, the assignment does not raise an error; it silently replaces `test_df`'s questions with `train_df`'s questions (row-aligned by position, not by content). Any downstream use of `test_df['prompt']` after this cell operates on the wrong text. This does not affect the TF-IDF validation score shown in Section 4, since that evaluation loop uses `val_df` rather than `test_df`, but it would corrupt TF-IDF test-time inference if that path were exercised on this `test_df` object later in a longer run. The fix is to call `test_df['prompt'].apply(...)` and assign back to `test_df['prompt']`.

6.2 Missing Standalone Model Metrics

As noted in Section 4, the notebook does not print a clearly separated final validation accuracy/F1/MAP@3 summary for DeBERTa alone or RoBERTa alone after their training cells complete, only Trainer's per-epoch logs (routed to Weights & Biases rather than the notebook's own stdout) and the combined ensemble metrics. Adding an explicit `calculate_metrics` call on each model's validation predictions, printed immediately after training, would make it possible to quantify the ensemble's marginal benefit over its best individual component.

6.3 Unjustified Ensemble Weights

As discussed in Section 5.3, the 0.70/0.30 split is not derived from a validation sweep in the visible cells. A brief grid search, or a validation-set logistic regression/optimization over the two probability vectors, would substantiate the choice.

6.4 Confounded Comparison Between Model 2 and Model 3

Model 2 (DeBERTa) and Model 3 (RoBERTa) differ simultaneously in architecture, fine-tuning strategy (full vs. LoRA), epoch count, and learning rate. This makes it impossible, from the notebook alone, to attribute either model's performance to any single one of these factors. Running a LoRA-adapted DeBERTa and/or a fully fine-tuned RoBERTa as additional ablations would isolate the effect of the adaptation method from the effect of the base architecture.

6.5 Reproducibility and Environment Notes

- The notebook installs and imports torchao but never uses it in any visible cell; a warning at import time indicates it partially failed to load due to a torch version mismatch (found 2.10.0+cu128, requires $\geq 2.11.0$). This dependency appears vestigial and could be removed.
- The Weights & Biases API key is retrieved through Kaggle's UserSecretsClient rather than hardcoded, which is good practice, though it does make the notebook non-portable outside the Kaggle environment without modification.
- `torch.backends.cudnn.deterministic=True` and `benchmark=False` are set, which is correct practice for reproducibility, though the notebook does not pin exact library versions (transformers, peft, torch) in a requirements file, so exact metric reproduction on a different environment is not guaranteed.

7. Conclusion

The notebook implements a coherent, well-commented progression from a weak lexical baseline to two independently trained transformer classifiers and a weighted probability ensemble. The final ensemble model achieves strong validation performance (97.50% accuracy, 0.9833 MAP@3), a substantial improvement over the TF-IDF baseline (12.50% accuracy, 0.2712 MAP@3). The architectural and training decisions documented throughout the notebook are individually reasonable and mostly explained in accompanying markdown notes.

The main outstanding issues are the confirmed test-set lowercasing bug in the TF-IDF stage (Section 6.1), the absence of standalone validation metrics for the two transformer models prior to ensembling (Section 6.2), and the unvalidated 70/30 ensemble weighting (Section 6.3). None of these compromise the final reported ensemble score, which is computed correctly from `val_df`, but addressing them would let the notebook's conclusions be verified and reproduced with more confidence.